tion code as *conditional code*, which forgoes the branching issue altogether—for smaller loops, at least. The ARM assembly code for the same operation goes thus:

```
loopCMPRi,Rj                    ;set condition "NE (not equal)" if (i != j)
                                ;          "GT (greater than)" if (i > j),
                                ;       or  "LT (less than)" if (i < j)
    SUBGT  Ri, Ri, Rj       ; if "GT", i = i - j;
    SUBLT  Rj, Rj, Ri       ; if "LT", j = j - i;
    BNE loop                ; if "NE", then loop
```

The result of the CMP (Compare) instruction is stored as a *condition flag* in the Current Processor State Register (CPSR) on the CPU for later reference.

Of the 32 bits that form the ARM's instruction, 28 are used for the actual instruction, and the remaining four carry the condition flag—this is ARM's own way of implementing RISC.

So when the SUBGT (Subtract if Greater Than) instruction is sent to the CPU, it isn't going to waste CPU cycles loading the condition flag and then checking it—the flag is right there, and depending on its contents, the instruction executes.

The final instruction tells the ARM to branch back to the label "loop" if i and j aren't equal. Notice that at no point here does the ARM have to jump to different memory locations—saving precious nanoseconds.

This clean, efficient way of going about things means that the ARM doesn't have to fight a Megahertz war—it can get more done for per clock cycle, so it can run on a lower clock frequency without too much loss. Lower clock frequencies translate to less heat generated within the chip, so the only thing heating up your ARM is probably the sunlight that you're basking in.

ARM doesn't have to fight a Megahertz war—it can get more done for per clock cycle, so it can run on a lower clock frequency without too much loss

Jargon Buster

Registers
Every processor has a number of *registers* built right on to the chip. These serve as temporary storage for data or instructions that come into the processor, or for storing important information like the processor's current state, or the memory address that it needs to access next. This is the fastest possible storage in your PC, so wasting it is a processor design no-no.

Assembly Language
When you give instructions to a microprocessor, you do so in Assembly Language. The instructions involve very granular operations in the processor—moving a byte of data from the main memory to the processor's internal registers, for instance. You write assembly code in the form of *mnemonics*—MOV, ADD, DIV and so on, which are then converted to hexadecimal *operation codes* or *op-codes*, which are finally given to the processor.

Microprograms
Microprograms (written in microcode) take op-codes and convert them into real processor actions. The ADD instruction, for instance, will bring numbers from the system's memory to the processor's registers before adding them—it's the microcode that implements this part. Microcode is hard-wired into the processor when it's designed, and optimising it for performance is right up there with the designers' biggest headaches of all time.

Coffee With Jazelle

We've no doubt talked about how Java works before, but here's a refresher anyway: when you compile a program written in Java, it doesn't get converted to the assembly code for the processor you're working on. Instead, it's translated to Java *bytecode*, which runs on the Java Virtual Machine (JVM). The JVM then translates the bytecode to assembly language for the processor. The advantage here is that unlike other languages, where you write different code for different platforms (very tedious), you write code just for the JVM. Now you have code that will run on any platform that has a JVM built for it!

The problem with Java, as should be fairly obvious, is that the time taken for the JVM to translate Java bytecode to assembly causes it to take a huge performance hit; this cannot do.

Enter Jazelle, ARM's way of combating this performance lag. The Jazelle Direct Bytecode eXecution (DBX) makes a JVM for ARM redundant—the ARM7, ARM10, and ARM11 families of processors can now run Java code directly! If you've ever experienced the tedium of using a Java application on your mobile phone, this is a ray of hope. To see the difference that Jazelle makes, watch the video at http://www.arm.com/products/esd/jazelle_home.html.

Bells And Whistles

RISC works wonderfully with data that comes in long, continuous streams—video and music, for example—making it a natural choice for any portable multimedia player (PMP). The iPod, among many others, sits on an ARM7-based processor, which is built specifically for multimedia. It's tiny, fast, and lasts hours on a single battery charge. Among the other licensees of the ARM, Freescale Semiconductor builds the i.MX, based on the ARM9 architecture. These processors feature on-board Multimedia Card (MMC) and Memory Stick controllers, a Multimedia Accelerator chip and a Bluetooth accelerator. The architecture supports Windows CE, Linux and Symbian, and you'll be seeing devices based on this in the next few months.

Pocket Powerplants

With the world going mobile and the new emphasis on performance per watt, we've started seeing new developments in processor design, which now permit even the good old x86 processor to be plonked into a mobile device. Samsung's Q1 Ultra runs a low-voltage Intel Core 2 Duo, and runs Vista Home Premium with no problems at all! The three-and-a-half hour battery life is disappointing, but the fact that a desktop-class processor can run in this environment is an indication of things to come.

But does that mean that the ARM will become obsolete? For a long time, no. There's hardly a competitor in the battery life department, and even when it is muscled out of the PDA / UMPC market, portable media players and smartphones will ensure that the ARMs still have punch. ■

nimish_chandiramani@thinkdigit.com